



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 4

“Contador de 0 a 9”

Grupo: 6CM1

Integrantes

- **García Escamilla Bryan Alexis**
- **Meléndez Macedonio Rodrigo**

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 13 de febrero de 2025

INTRODUCCIÓN

Contador de 0 a 9

Un contador de 0 a 9, técnicamente conocido como Contador de Década o Contador BCD (Binary Coded Decimal), es un dispositivo lógico que recorre exactamente diez estados antes de reiniciarse. Es la base fundamental de cualquier sistema que interactúe con humanos, ya que nuestro sistema numérico es decimal.

Lógica

Un contador binario natural de 4 bits cuenta normalmente del 0 al 15 (16 estados). Para obtener un contador de 0 a 9, es necesario truncar la secuencia.

- **El límite binario:** El número 9 en binario es **1001**. El siguiente número sería el 10, que es **1010**.
- **La detección del estado:** El circuito utiliza compuertas lógicas (generalmente una compuerta NAND) conectadas a los bits que representan el número 10. Cuando el contador intenta pasar de 9 a 10, la compuerta detecta el **1010** y envía una señal instantánea al pin de Reset de todos los flip-flops.
- **El resultado:** El número 10 es tan efímero (dura nanosegundos) que el ojo humano y los circuitos de salida solo perciben el salto del 9 de vuelta al 0.

Diagrama de estados y transiciones

En la teoría de circuitos secuenciales, el contador de década se define por su módulo (MOD-10).

- **Estados válidos:** 0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 1001.
- **Estados inválidos:** Del 1010 al 1111. Si por algún ruido eléctrico el contador cae en uno de estos estados, el diseño debe ser capaz de retornar a la secuencia válida.

Configuración en cascada

Para contar más allá del 9 (por ejemplo, hasta 99 o 999), los contadores de década utilizan una salida especial llamada Terminal Count (TC) o Carry Out.

- Cuando el contador llega a su último estado (9) y recibe el siguiente pulso de reloj, emite un pulso por este pin.
- Este pulso sirve como "reloj" para el siguiente contador (el de las decenas). Así, cuando las unidades pasan de 9 a 0, las decenas aumentan en 1.

Diferencia entre Contador de Década y BCD

Aunque a menudo se usan como sinónimos, hay una diferencia sutil en la teoría:

- **Contador BCD:** Específicamente entrega 4 salidas que representan el código binario del 0 al 9. Es el que conectas a un convertidor para display de 7 segmentos.
- **Contador de Década:** Puede referirse a cualquier contador con 10 estados, incluso aquellos que tienen 10 salidas individuales donde solo una está activa a la vez.

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 4, el circuito a armar es el siguiente:

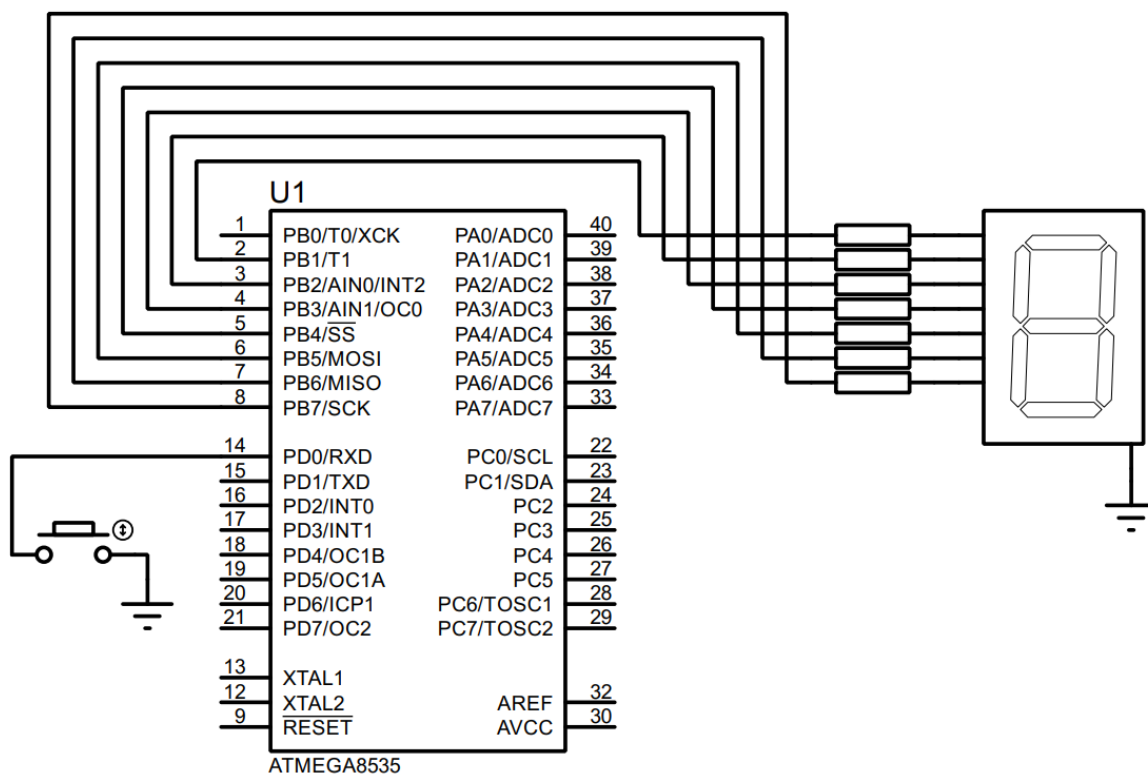


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen del circuito, el registro B se configura como de salida (PB1 – PB7) mientras que el registro D como de entrada (PD0), activando la resistencia de pull-up.

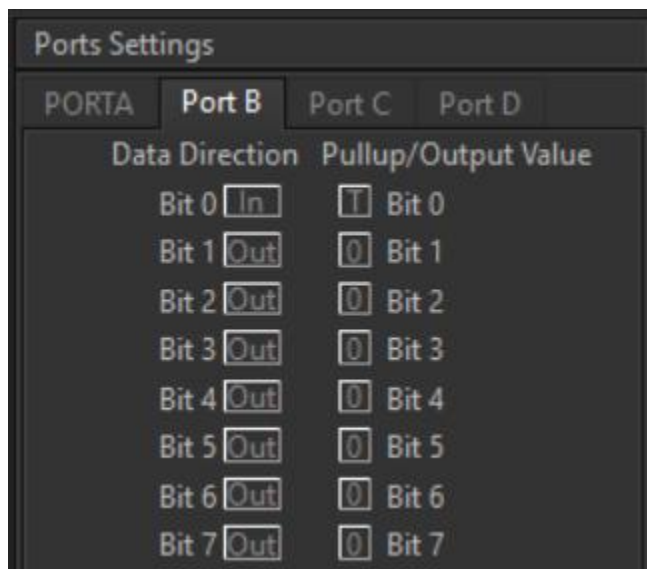


Imagen 2. Configuración del puerto B.

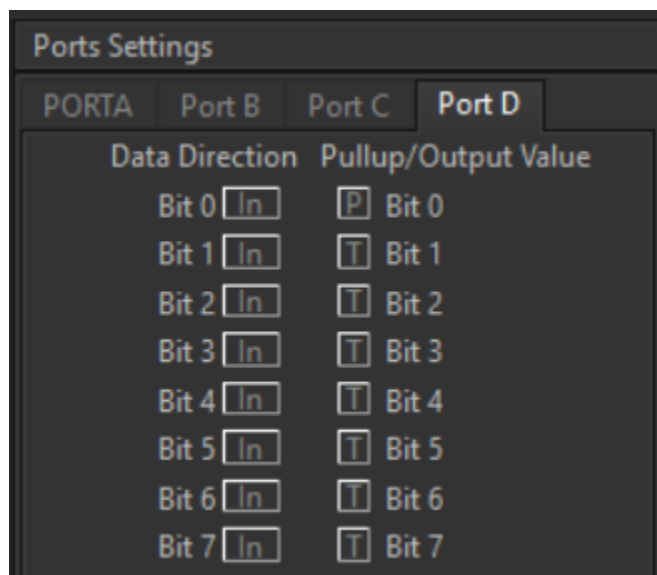


Imagen 3. Configuración del puerto D.

Para poder hacer el contador es necesario interpretar el número correspondiente del display como un número hexadecimal, para ello se realizó la siguiente tabla, tomando en cuenta lo que se indicó en la práctica 3:

Número Display	Combinaciones								Valor hexadecimal
	g	f	e	d	c	b	a		
0	0	1	1	1	1	1	1	0	0x7e
1	0	0	0	0	1	1	0	0	0x0c
2	1	0	1	1	0	1	1	0	0xb6
3	1	0	0	1	1	1	1	0	0x9e
4	1	1	0	0	1	1	0	0	0xcc

5	1	1	0	1	1	0	1	0	0xda
6	1	1	1	1	1	0	1	0	0xfa
7	0	0	0	0	1	1	1	0	0x0e
8	1	1	1	1	1	1	1	0	0xfe
9	1	1	0	1	1	1	1	0	0xde
A	1	1	1	0	1	1	1	0	0xee
B	1	1	1	1	1	0	0	0	0xf8
C	0	1	1	1	0	0	1	0	0x72
D	1	0	1	1	1	1	0	0	0xbc
E	1	1	1	1	0	0	1	0	0xf2
F	1	1	1	0	0	0	1	0	0xe2

Tabla 1. Representaci3n hexadecimal para los n3meros del 0 al 9 y del A a la F en hexadecimal.

C3digo del circuito de CodeVision

```

1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4  Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Pr3ctica 4 'Contador de 0 a 9'
8  Version : 1.0
9  Date    : 11/02/2026
10 Author  : Garc3a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type           : ATmega8535
16 Program type        : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model         : Small
19 External RAM size    : 0
20 Data Stack size      : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25 #define BOTON PIND.0

```

```

27 // Declare your global variables here
28
29 void main(void) {
30     // Declare your local variables here
31     unsigned char contador;
32     const char bcd[10] = {0x7e, 0x0c, 0xb6, 0x9e, 0xcc, 0xda, 0xfa, 0x0e, 0xfe, 0xde};
33
34     // Input/Output Ports initialization
35     // Port A initialization
36     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
37     DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
38     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
39     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
40
41     // Port B initialization
42     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
43     DDRB=(1<<ddb7) | (1<<ddb6) | (1<<ddb5) | (1<<ddb4) | (1<<ddb3) | (1<<ddb2) | (1<<ddb1) | (1<<ddb0);
44     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
45     PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
46
47     // Port C initialization
48     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
49     DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
50     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
51     PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);
52
53     // Port D initialization
54     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
55     DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
56     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=P
57     PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (1<<PORTD0);
58
59     // Timer/Counter 0 initialization
60     // Clock source: System Clock
61     // Clock value: Timer 0 Stopped
62     // Mode: Normal top=0xFF
63     // OC0 output: Disconnected
64     TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
65     TCNT0=0x00;
66     OCR0=0x00;
67
68     // Timer/Counter 1 initialization
69     // Clock source: System Clock
70     // Clock value: Timer1 Stopped
71     // Mode: Normal top=0xFFFF
72     // OC1A output: Disconnected
73     // OC1B output: Disconnected
74     // Noise Canceler: Off
75     // Input Capture on Falling Edge
76     // Timer1 Overflow Interrupt: Off
77     // Input Capture Interrupt: Off
78     // Compare A Match Interrupt: Off
79     // Compare B Match Interrupt: Off
80     TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
81     TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
82     TCNT1H=0x00;
83     TCNT1L=0x00;
84     ICR1H=0x00;
85     ICR1L=0x00;
86     OCR1AH=0x00;
87     OCR1AL=0x00;
88     OCR1BH=0x00;
89     OCR1BL=0x00;

```



```

91 // Timer/Counter 2 initialization
92 // Clock source: System Clock
93 // Clock value: Timer2 Stopped
94 // Mode: Normal top=0xFF
95 // OC2 output: Disconnected
96 ASSR=(0<<AS2);
97 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
98 TCNT2=0x00;
99 OCR2=0x00;
100
101 // Timer(s)/Counter(s) Interrupt(s) initialization
102 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
103
104 // External Interrupt(s) initialization
105 // INT0: Off
106 // INT1: Off
107 // INT2: Off
108 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
109 MCUCSR=(0<<ISC2);
110
111 // USART initialization
112 // USART disabled
113 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);
114
115 // Analog Comparator initialization
116 // Analog Comparator: Off
117 // The Analog Comparator's positive input is
118 // connected to the AIN0 pin
119 // The Analog Comparator's negative input is
120 // connected to the AIN1 pin
121 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
122 SFIOR=(0<<ACME);
123
124 // ADC initialization
125 // ADC disabled
126 ADSCRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);
127
128 // SPI initialization
129 // SPI disabled
130 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
131
132 // TWI initialization
133 // TWI disabled
134 TWCR=(0<<TWIEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

```

```

136 while (1) {
137     // Place your code here
138     if (!BOTON) {
139         contador++;
140     }
141
142     if (contador == 10) {
143         contador = 0; // Reiniciar conteo
144     }
145
146     PORTB = bcd[contador];
147 }
148 }

```

Circuito físico

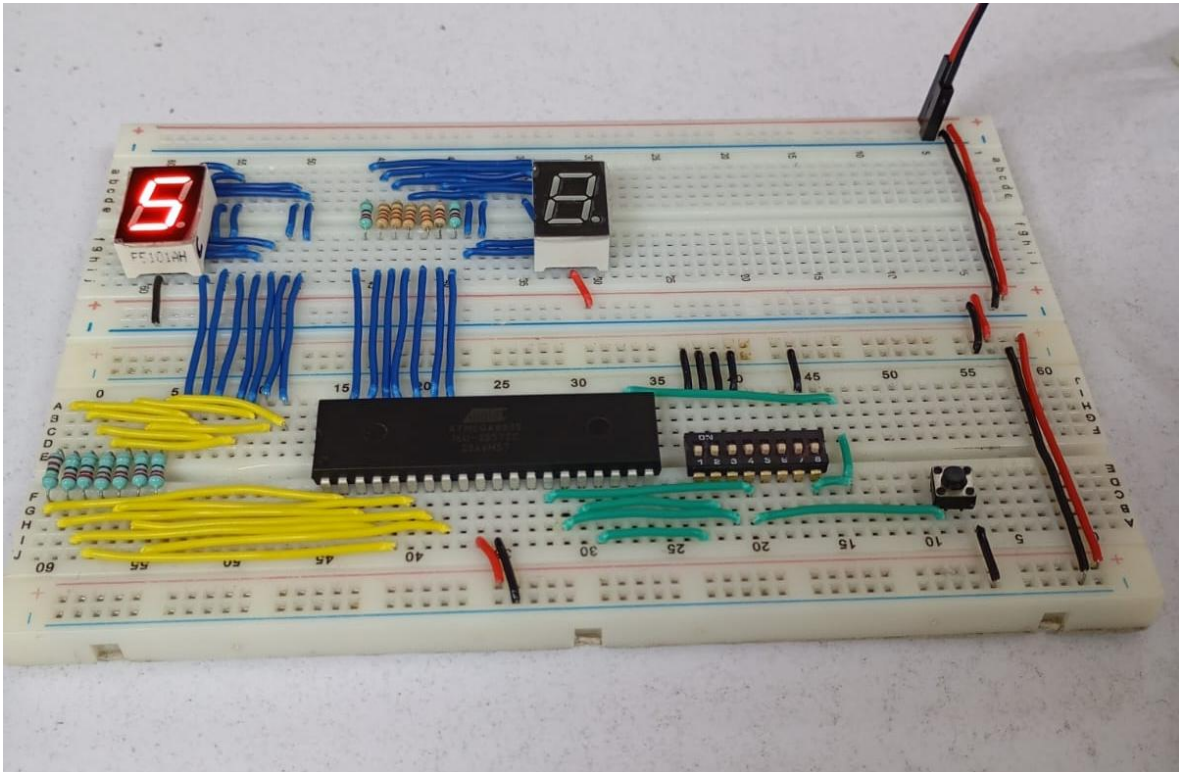


Imagen 3. Circuito físico de la práctica 4.

CONCLUSIÓN

- **García Escamilla Bryan Alexis**

En esta práctica se utiliza por primera vez un botón mecánico, el cual trae consigo algunos detalles a los que se le deben de poner atención para su uso en futuras prácticas. Por el momento en el código solo se detecta cuando el botón es presionado, es decir cuándo el bit del registro es cero y así poder incrementar el contador, sin embargo, existe algo llamado 'rebote' que se refiere a que al presionar el botón y soltarlo se producen ciertas vibraciones las cuales el ATMEGA interpreta como ceros lógicos, incrementando el contador rápidamente dando el efecto de que pareciese que el contador se salta algunos números.

En la siguiente práctica se intentará solucionar el problema detectando el flanco de bajada del botón.

- **Meléndez Macedonio Rodrigo**

Para el desarrollo de esta práctica se desarrolló un contador de 0 a 9 que incrementa su valor cada vez que nosotros presionamos el botón mecánico desde el circuito armado. Del mismo modo que las anteriores prácticas, CodeVisionAVR ofrece muchas facilidades al momento de implementar el código en C y al igual que la anterior práctica, utilizamos un arreglo para almacenar el valor en binario de los valores del 0 a 9 respectivamente. El código solo se encarga de aumentar el valor del contador con "++" y cuando llega el número 10, este

contador se reinicia en 0, repitiendo el ciclo cada vez que nosotros presionamos el botón. Sin embargo, esta práctica tiene un gran fallo que está hecho a propósito y es que al momento de presionar el botón este puede tener “rebotes”, los cuales el microcontrolador ATMEGA8535 los puede interpretar como ceros lógicos y eso hace que el valor del contador aumente erróneamente a pesar de que nosotros solo presionamos el botón una sola vez. Este fallo se arreglará poco a poco más adelante con futuras implementaciones.

BIBLIOGRAFÍA

- (s.f.). *Contadores de Década*. Electrónica Completa. Recuperado el 10 de febrero de 2026. <https://electronicaCompleta.com/contador-de-decada/>